

Implementing Classic Subword Tokenization Algorithms: BPE and WordPiece

BPE and WordPiece

Mohamed Ibrar UL HASSAN

Temidayo James AFOLABI

Lama YASSIN

Supervised by Professor Marie CANDITO

Plan

- | | |
|----------------------|---|
| 1. Why tokenization? | <i>The problem we are solving</i> |
| 2. BPE | <i>The frequency-based algorithm</i> |
| 3. WordPiece | <i>The information-based algorithm</i> |
| 4. Our experiments | <i>Five experiments on German Wikipedia</i> |
| 5. Key results | <i>What the numbers tell us</i> |
| 6. Conclusion | <i>What we learned</i> |

Why tokenization?

A neural network only understands numbers, not letters. Before any model can process a sentence, it must be turned into a list of numbers. The component that does this is called a tokenizer.

Characters

Too granular

T, h, e, c, a, t, s ...

The model must learn that t-h-e means "the" on its own.
Sequences become very long.
Useful patterns disappear.

Words

OOV problem

"cat" is one number, "cats" is another.

New words crash the model.
Vocabulary would need to be infinite.

Subwords

What BPE and WordPiece do

"cats" = cat + s
"running" = run + ning

LEGO-brick approach. Nothing falls out of vocabulary.

BPE (Byte Pair Encoding)

Sennrich et al., 2016 | Used by GPT and RoBERTa

The rule is simple: always merge the most frequent pair of adjacent symbols.

- 1 Start by splitting every word into individual characters.
- 2 Count how often each pair of adjacent symbols appears in the corpus.
- 3 Take the most frequent pair and merge it into one new symbol.
- 4 Add this new symbol to the vocabulary.
- 5 Repeat until the vocabulary reaches the target size.

The output is an ordered list of merge rules. This order makes tokenization deterministic: the same word always gives the same tokens.

WordPiece

Schuster and Nakajima, 2012 | Used by BERT and DistilBERT

WordPiece does not merge the most frequent pair. It merges the most informative pair.

$$\text{score}(A, B) = \text{frequency}(AB) / (\text{frequency}(A) \times \text{frequency}(B))$$

What this means in plain terms:

The numerator rewards pairs that appear together often. The denominator punishes pairs where each piece is already very common on its own. WordPiece does not merge a pair just because it is frequent. It merges a pair because the two pieces appear together more than you would expect.

Example: in German, "d" and "e" are both very common, so BPE merges "de" early. WordPiece does not, because seeing "d" followed by "e" is not surprising. WordPiece prefers a pair like "qu", where "q" almost never appears without "u".

WordPiece also uses a ## prefix to mark pieces that continue a word, not start one.

Example: spielen becomes spiel + ##en

Our experimental setup

Corpus

We used 10% of German Wikipedia (snapshot 20231101.de, via HuggingFace). We chose German deliberately because it is morphologically rich: long compound words like Bundesrepublik, productive suffixes like -lich and -keit. After cleaning, roughly 3.9 million unique words with counts.

Gold standard and metrics

We built a morphological gold standard by hand: 20 German words with known correct splits. We included verbs, compound nouns, words with prefixes and suffixes, and an inflection family (trinke, trinkst, trinkt, getrunken). We also used spaCy (German model) as an independent automatic cross-check, as the project brief suggested.

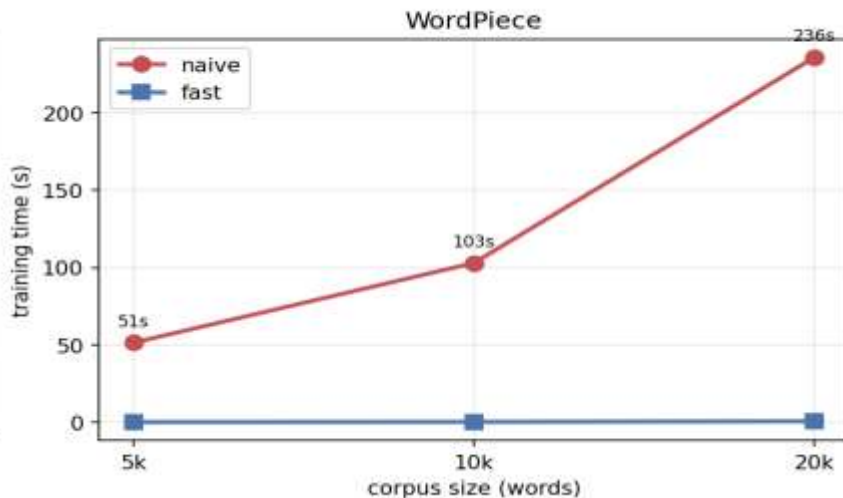
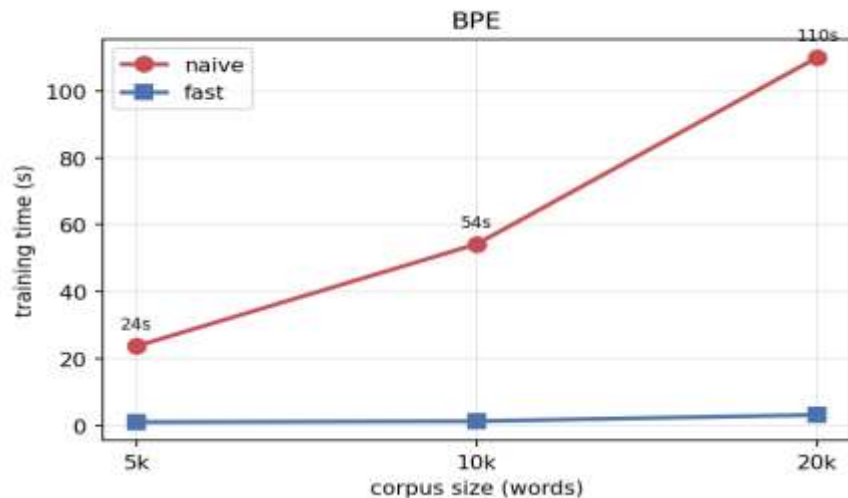
Two metrics: exact match (is the split exactly right?) and morpheme recall (what fraction of expected morphemes appear?).

Experiment 1 : training speed

Naive implementation vs fast implementation

The naive approach re-reads the entire corpus on every merge. Our fast version uses an inverse index, incremental counts, and a lazy heap to avoid this.

Training time: naive vs fast (lower is better)



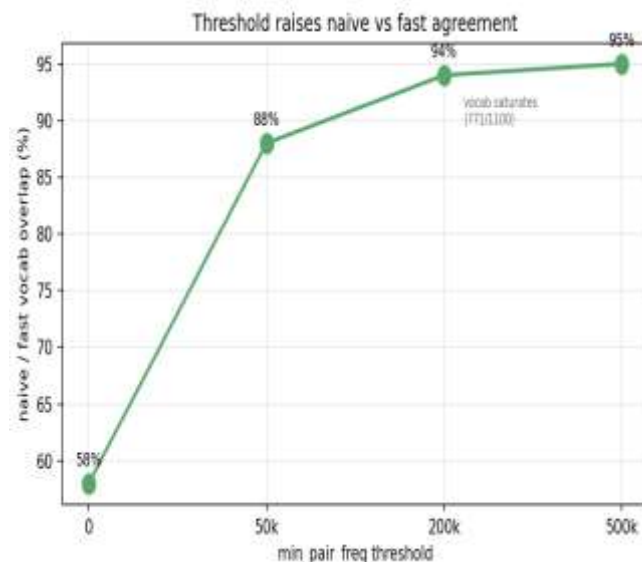
The naive cost grows linearly with data. The fast cost stays nearly flat. This difference in shape is what matters.

Experiment 2: the WordPiece frequency threshold

Without filtering, WordPiece assigns high scores to very rare pairs, producing junk merges like `##wj`. The standard fix is to ignore pairs that appear fewer than a threshold number of times.

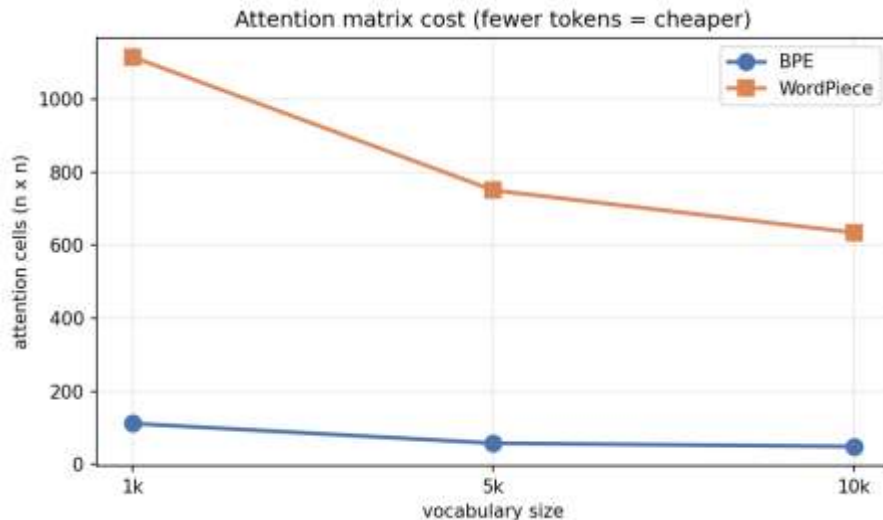
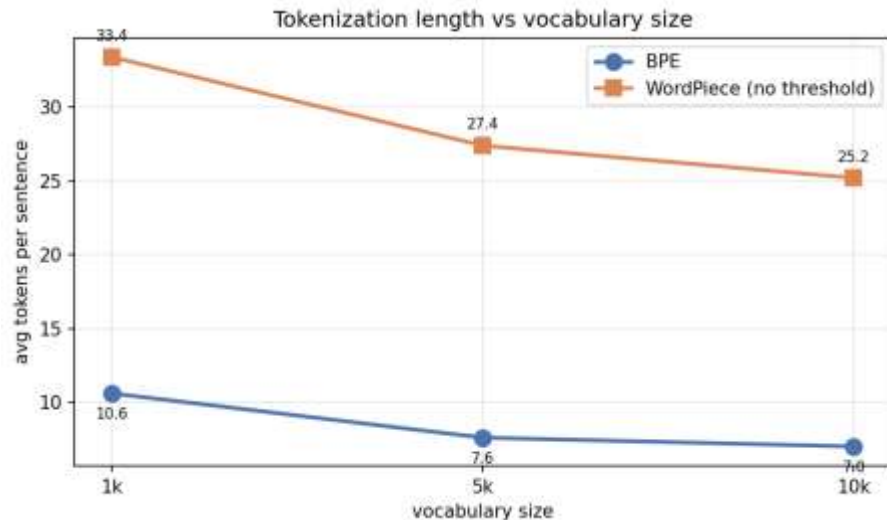
The threshold is a genuine trade-off: it improves merge quality but the vocabulary stops growing once all remaining pairs are filtered out. This is saturation.

Threshold	Agreement	Vocab reached
0 (off)	58%	1100 / 1100
50 000	88%	1100 / 1100
200 000	94%	771 / 1100 (saturated)
500 000	95%	357 / 1100 (saturated)



Experiments 3 to 5 therefore run WordPiece with the threshold off, to study vocabulary size effects independently.

Experiments 3 and 4 : vocabulary size and attention cost



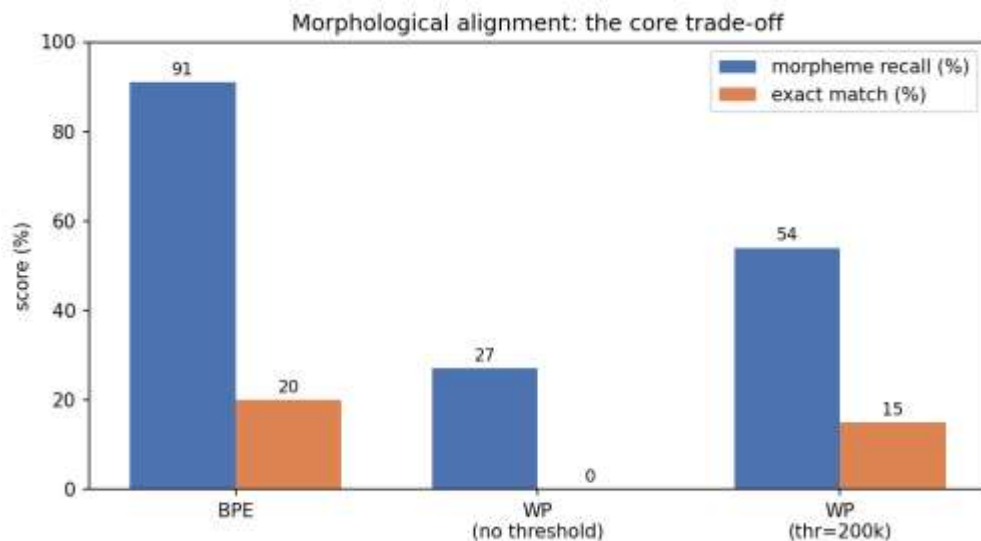
BPE consistently produces more compact tokenizations. This is not a bug. It is a direct consequence of WordPiece's scoring: it builds long common pieces more slowly.

Why does token count matter?

The attention mechanism grows in cost with the square of the sequence length ($n \times n$ matrix). At vocabulary size 1000, WordPiece implies an attention matrix nearly ten times larger than BPE on the same sentences. Tokenizer choice is an efficiency decision.

Experiment 5 : morphological evaluation

How well do the produced tokens align with the actual morphemes of the language?



Configuration	Recall	Exact
BPE (no tuning)	0.91	4/20
WP (no threshold)	0.27	0/20
WP (threshold 200k)	0.52	3/20

spaCy cross-check (independent):

BPE kept 2/3 lemma families intact vs WordPiece 0/3. Recall: BPE 0.62 vs WordPiece 0.14. Two independent methods reach the same conclusion.

BPE aligns with German morpheme boundaries almost without trying. WordPiece needs tuning and still trails behind.

Problems we found and fixed

The stale-heap bug in fast WordPiece

WordPiece's score depends on a global denominator that changes whenever any merge happens elsewhere. Our fast version detected stale heap entries but discarded them instead of re-inserting with the correct score. A pair could therefore be lost forever, even if it was the best remaining choice.

Fix: when a stale entry is popped, re-insert it with its refreshed score. After the fix, naive and fast agreement rose from about 19% to about 55%.

This also explains why WordPiece is harder to make fast than BPE. BPE's selection depends only on local pair counts. WordPiece's does not.

Missing training cells in the notebooks

The committed notebooks were missing the cells that actually call the training functions. A fresh checkout would fail immediately. We rebuilt these cells so every reported speed number is computed live when the notebook runs, not copied in by hand.

Limitations

Our gold standard is small.

Twenty words plus one inflection family is enough to show the direction clearly, but not enough for a full statistical evaluation. That said, our cross-check with spaCy as a second independent source gave the same result.

Our vocabulary sizes are modest.

We worked with vocabularies up to 10 000. Production tokenizers like BERT use 30 000 trained on far more data. Neither algorithm is operating at full strength here.

The inflection test defeats both algorithms.

Neither BPE nor WordPiece cleanly isolates the root trink across trinke, trinkst, trinkt and getrunken. This is not a flaw in our implementation. It is an honest limit of subword tokenization itself. These methods are statistical, not linguistic.

WordPiece's exact training procedure is unknown.

Google never released the precise algorithm used for BERT. We implemented the publicly documented approximation. Some divergence is expected.

Conclusion

- **The fast implementations work.**

Naive training time grows with the corpus. Fast training time stays nearly flat. The speedup is 21 to 40 times for BPE and 220 to 310 times for WordPiece.

- **The two algorithms produce different vocabularies.**

BPE optimises frequency. WordPiece optimises informativeness. On the same data, they make different choices and produce different tokens.

- **On German, BPE aligns with morpheme boundaries almost without trying.**

It achieves 0.91 morpheme recall with no tuning. WordPiece needs a frequency threshold to approach this, and even then only reaches 0.52 with a saturated vocabulary.

- **Tokenizer choice has computational consequences.**

BPE produces fewer tokens per sentence, which means a smaller attention matrix. At vocabulary size 1000, WordPiece implies roughly ten times more attention cost than BPE.

- **Both methods are statistical, not linguistic.**

Neither algorithm truly understands morphology. The inflection test reveals this clearly. This is an honest limit of the subword tokenization paradigm itself.

Thank you

Mohamed Ibrar UL HASSAN | Temidayo James AFOLABI | Lama YASSIN

Supervisor: Professor Marie CANDITO

NLP Project | Master 1 | Université Paris Cité | 2025-2026



Sennrich et al. (2016) | Schuster and Nakajima (2012) | HuggingFace LLM Course, Ch. 6 | German Wikipedia 20231101.de